

A close-up photograph of a person's hand in a dark red sweater reaching over a Go board. The board is a light-colored stone surface with a grid of lines. Black and white Go stones are scattered across the board. To the right of the board is a white plastic bucket. The background is a blurred green landscape. The text "Simulation Using SimPy" is overlaid in the center of the image.

Simulation Using SimPy

Objectives

- Perform basic simulation using SimPy.
- Simulate ATM line.
- Discuss the process in creating the simulation.

Four Building Blocks of SimPy

- Environment
- Event
- Process
- Resource

Environment

- Where everything resides.
- Simulation environment manages the simulation time as well as the scheduling and processing of events.

SimPy offers:

- Environment-simulate “as fast as possible”
- RealtimeEnvironment-synchronized with wall-clock time.

Event

- Example: ATM Line
- Event: Arrival or Departure of a person

Events in SimPy:

A. Timeout

- Basic event type
- Let's time pass
- Used to model phenomena such as waiting time and service times



Event

- Example: ATM Line
- Event: Arrival or Departure of person

Events in SimPy:

B. Asynchronous & Synchronous

Used to model:

- *Interactions*
- *Interruptions*



Event

- Example: ATM Line
- Event: Arrival or Departure of person

Events in SimPy:

C. Condition Event

- Waits for the condition to be satisfied
- Make use of logical operators such as “AND” and “OR”



Processes

Processes are active elements of SimPy

Used to model active entities

Example: A person going to the ATM and requesting one of the machine's takeout money is a process

In SimPy, you would write a process in which you would request for resources or schedule an event such as **timeout** event to simulate waiting.

Processes can be:

- Delayed for fixed or random times
- Queued at a resource facility
- Interrupted by others
- Interacted with others



Processes

For example:

- *A person going to the ATM to withdraw is process 1*
- (Assume the ATM has insufficient amount of money)*
- *This means that process 1 has to wait*
 - *The banker putting in more money is process 2*



Resources

Types of Resources:

A. Resource

B. Store

C. Container

- Resources are entities that can be ceased and released by processes.
- Store

How SimPy is Used in Modeling and Simulation

1. Define the Simulation Environment

The environment acts as the “clock” of the simulation. It keeps track of time and schedules events.

2. Model Processes as Generators

Each entity in the system (for example, a patient, customer, or machine) is modeled as a Python function using yield. These processes describe behavior over time.

How SimPy is Used in Modeling and Simulation

3. Use Resources and Queues

SimPy provides built-in objects such as `Resource`, `PriorityResource`, and `Container` to simulate limited resources like doctors, servers, or machines.

4. Define Events and Delays

Processes wait for events using statements like `yield env.timeout(5)` to simulate delays or processing times.

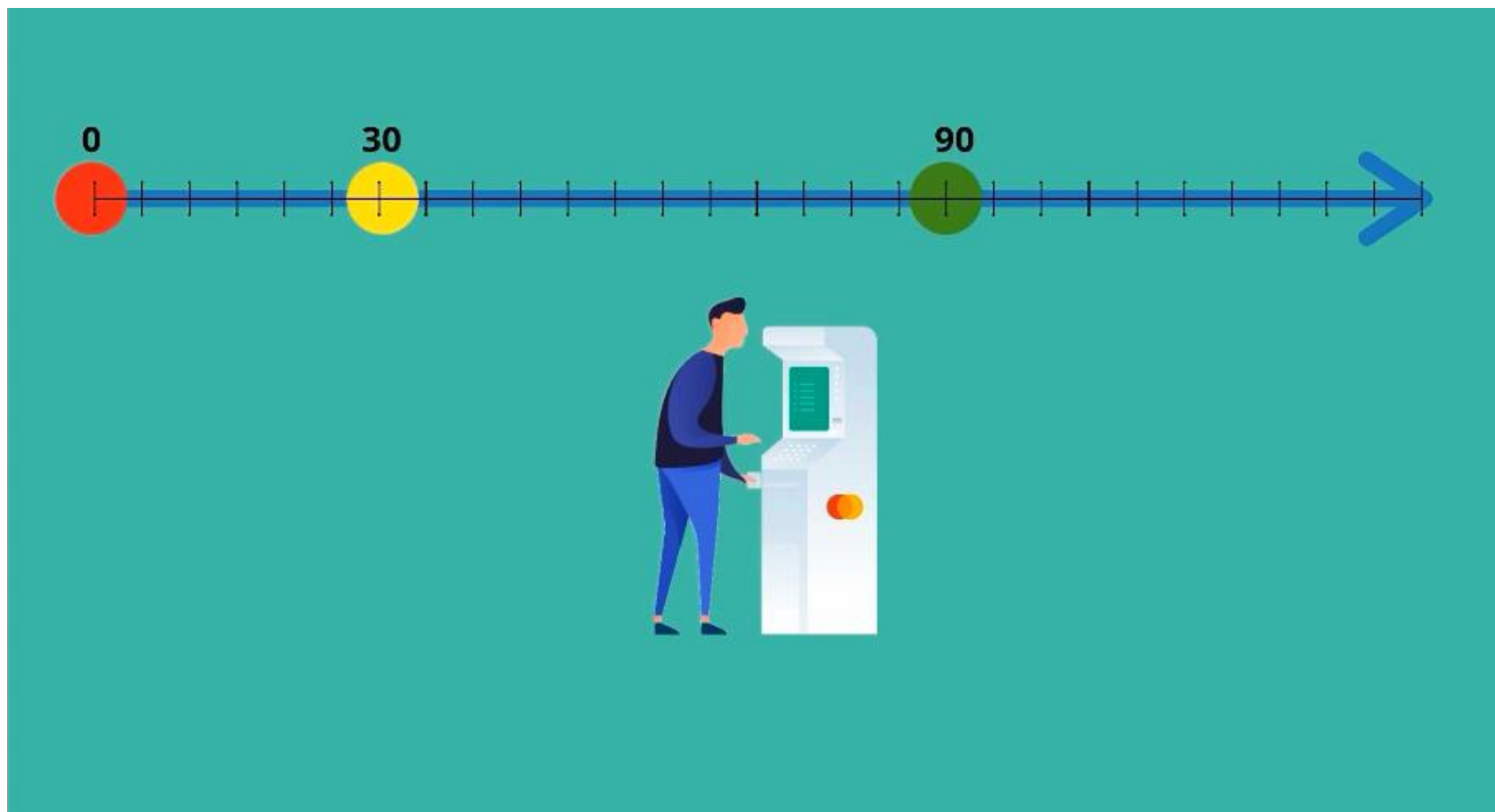
5. Run the Simulation

The simulation is executed for a specified amount of simulated time, allowing you to observe outcomes.

Visit to the ATM

- Model an ATM where a customer arrives. He spends 30 seconds to enter his details (such as PIN and amount) and it takes another 60 seconds to get the cash.

Visit to the ATM



Visit to the ATM

- Event

Syntax (defining timeout)

```
yield.env.timeout(30)
```

```
yield.env.timeout(60)
```

Yield can be interpreted as ‘wait for’

Visit to the ATM

- Environment

```
env=simpy.Environment()  
env.process(customer(env))  
env.run()
```

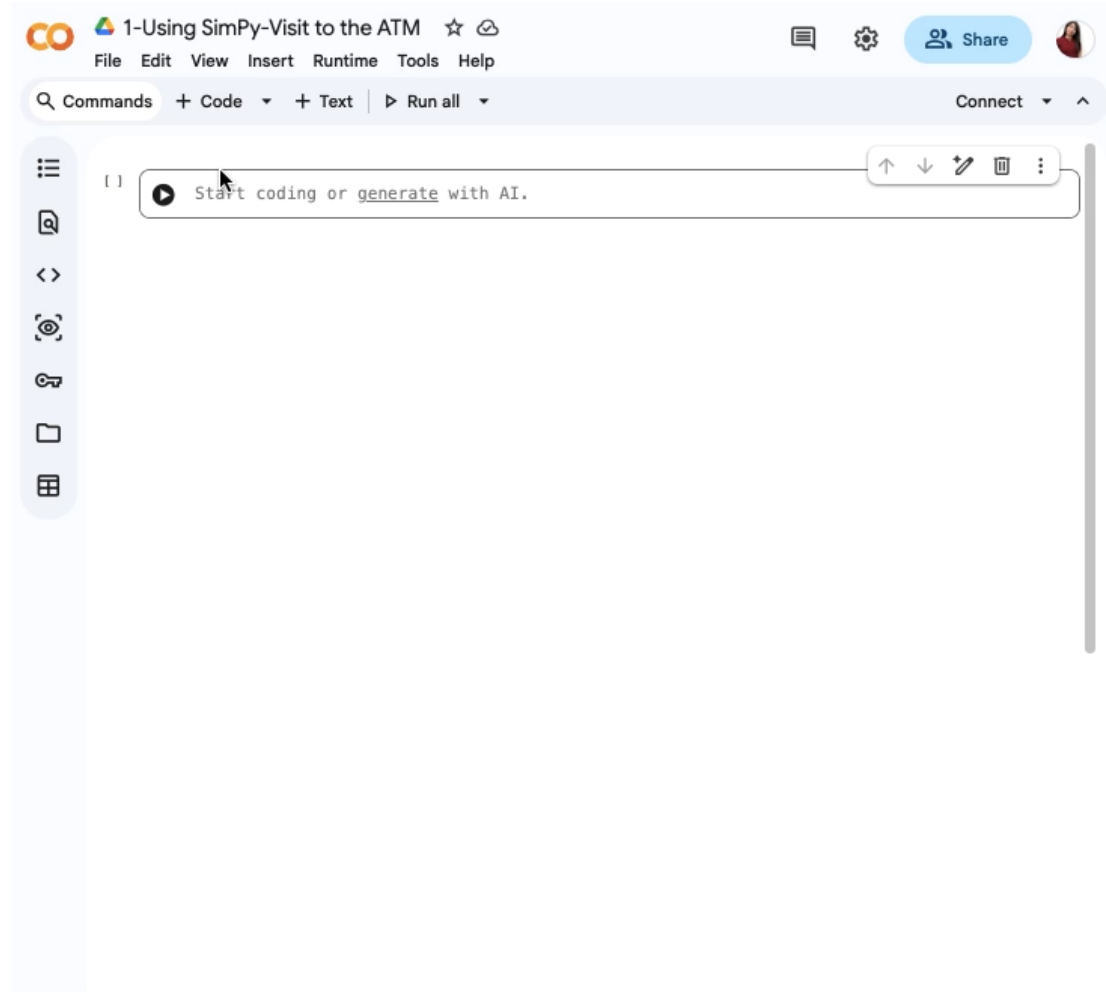
Visit to the ATM

- Process

```
def customer(env):  
    yield env.timeout(30) #wait for 30 seconds  
    print(f'Details entered by time{env.now}')  
    yield env.timeout(60) # wait for 60 seconds  
    print(f 'Cash retrieved at time {env.now}')
```

Coding

Installation



Installation Code

[1]
✓ 8s

▶ !pip install simpy

▼

```
... Collecting simpy
      Downloading simpy-4.1.1-py3-none-any.whl.metadata (6.1 kB)
      Downloading simpy-4.1.1-py3-none-any.whl (27 kB)
      Installing collected packages: simpy
      Successfully installed simpy-4.1.1
```


Code and Output: Visit to the ATM

[3]

✓ 0s

```
import simpy

env=simpy.Environment()

def customer(env):
    yield env.timeout(30)
    print(f'Details entered at time: {env.now}')
    yield env.timeout(60)
    print(f'Cash retrieved at time: {env.now}')

env.process(customer(env=env))

env.run()
```

✓

```
Details entered at time: 30
Cash retrieved at time: 90
```

Code in detail:

Creating the simulation environment

```
env = simpy.Environment()
```

- **env** acts as the ***clock and scheduler of the simulation***. It keeps track of the current simulation time (`env.now`) and manages all events.

Code in detail:

```
def customer(env) :
```

- **This defines a process (or generator function) called customer.**
- In SimPy, processes are written as Python generator functions using yield.
- This function describes the sequence of actions a “customer” will take over time.

Code in detail:

- `yield env.timeout(30)`

This line tells the simulation:

- “Pause this process for 30 (seconds) time units.”
- The process is inactive during this period.
- The simulation clock advances while waiting.
- So, nothing happens for this customer until time = 30.

Code in detail:

- `print(f'Details entered at time: {env.now}')`

After the 30 (seconds) time units pass:

- The process resumes.
- It prints the current simulation time using `env.now`.

Code in detail:

- `yield env.timeout(60)`

Now the process pauses again:

- It waits another 60 (seconds)time units.
- Since the current time is 30, the next action will occur at time = 90.

Code in detail:

```
env.process (customer (env=env) )
```

- This line registers the process with the environment.
- `customer(env)` creates the process.
- `env.process(...)` schedules it for execution.
- ***Without this, the function would never run in the simulation.***

Code in detail:

- **`env.run()`**
- This starts the simulation.
- The environment begins executing all scheduled processes.
- It runs until there are no more events left.

In Summary:

What actually happens during execution:

1. Simulation starts at time = 0
2. Customer process begins
3. Waits 30 (seconds) time units
4. Prints: Details entered at time: 30
5. Waits another 60 (seconds) time units
6. Prints: Cash retrieved at time: 90
7. Simulation ends (no more events)

Simulation: Visit to the ATM (Multiple Customers)

- Model an ATM where multiple customers arrive. Each one of them spends 30 seconds to enter his details (such as PIN and amount) and it takes another 60 seconds to get the cash.

Code: Constant Arrival Time

```
import simpy
import random

def customer(env,name):
    print(f'{name}: Arrives at time {env.now: .2f}')
    yield env.timeout(30)
    print(f'{name}: Details entered at time: {env.now: .2f}')
    #print(f' {name}: Details entered at time: {env.now: .2f}')
    yield env.timeout(60)
    print(f'{name}: Cash retrieved at time: {env.now: .2f}')

def customer_generator(env):
    cust_number=1
    while True:
        random_inter_arrival_time=random.uniform(1,3)*60
        yield env.timeout(random_inter_arrival_time)
        env.process(customer(env=env, name=f"customer {cust_number}"))
        cust_number+=1

env=simpy.Environment()

env.process(customer_generator(env=env))

env.run(until=10*60) #until 10 minutes
```

Code: Constant Arrival Time

Code Output 1

```
[9] import simpy
    import random

def customer(env,name):
    print(f'{name}: Arrives at time {env.now: .2f}')
    yield env.timeout(30)
    print(f'{name}: Details entered at time: {env.now: .2f}')
    yield env.timeout(60)
    print(f'{name}: Cash retrieved at time: {env.now: .2f}')

def customer_generator(env):
    cust_number=1
    while True:
        random_inter_arrival_time=random.uniform(1,3)*60
        yield env.timeout(random_inter_arrival_time)
        env.process(customer(env=env, name=f"customer {cust_number}"))
        cust_number+=1

#random.seed(2)

env=simpy.Environment()

env.process(customer_generator(env=env))

env.run(until=10*60) #until 10 minutes

... customer 1: Arrives at time 148.32
customer 1: Details entered at time: 178.32
customer 1: Cash retrieved at time: 238.32
customer 2: Arrives at time 288.68
customer 2: Details entered at time: 318.68
customer 2: Cash retrieved at time: 378.68
customer 3: Arrives at time 385.66
customer 3: Details entered at time: 415.66
customer 3: Cash retrieved at time: 475.66
customer 4: Arrives at time 518.37
customer 4: Details entered at time: 548.37
```

Code Output 2

Random Arrival Time

```
[10] import simpy
      import random

def customer(env,name):
    print(f'{name}: Arrives at time {env.now: .2f}')
    yield env.timeout(30)
    print(f'{name}: Details entered at time: {env.now: .2f}')
    yield env.timeout(60)
    print(f'{name}: Cash retrieved at time: {env.now: .2f}')

def customer_generator(env):
    cust_number=1
    while True:
        random_inter_arrival_time=random.uniform(1,3)*60
        yield env.timeout(random_inter_arrival_time)
        env.process(customer(env=env, name=f"customer {cust_number}"))
        cust_number+=1

#random.seed(2)

env=simpy.Environment()

env.process(customer_generator(env=env))

env.run(until=10*60) #until 10 minutes

... customer 1: Arrives at time 129.74
customer 1: Details entered at time: 159.74
customer 2: Arrives at time 208.75
customer 1: Cash retrieved at time: 219.74
customer 2: Details entered at time: 238.75
customer 2: Cash retrieved at time: 298.75
customer 3: Arrives at time 320.43
customer 3: Details entered at time: 350.43
customer 3: Cash retrieved at time: 410.43
customer 4: Arrives at time 427.65
customer 4: Details entered at time: 457.65
customer 4: Cash retrieved at time: 517.65
customer 5: Arrives at time 574.43
```

Code with random seed()

[7]
✓ 0s



```
import simpy
import random
```

```
def customer(env,name):
    print(f'{name}: Arrives at time {env.now: .2f}')
    yield env.timeout(30)
    print(f'{name}: Details entered at time: {env.now: .2f}')
    #print(f '{name}: Details entered at time: {env.now: .2f}')
    yield env.timeout(60)
    print(f'{name}: Cash retrieved at time: {env.now: .2f}')

def customer_generator(env):
    cust_number=1
    while True:
        random_inter_arrival_time=random.uniform(1,3)*60
        yield env.timeout(random_inter_arrival_time)
        env.process(customer(env=env, name=f"customer {cust_number}"))
        cust_number+=1

random.seed(2)

env=simpy.Environment()

env.process(customer_generator(env=env))

env.run(until=10*60) #until 10 minutes
```

Code with random seed()

OUTPUT

```
[8]
✓ 0s  import simpy
import random

def customer(env,name):
    print(f'{name}: Arrives at time {env.now: .2f}')
    yield env.timeout(30)
    print(f'{name}: Details entered at time: {env.now: .2f}')
    yield env.timeout(60)
    print(f'{name}: Cash retrieved at time: {env.now: .2f}')

def customer_generator(env):
    cust_number=1
    while True:
        random_inter_arrival_time=random.uniform(1,3)*60
        yield env.timeout(random_inter_arrival_time)
        env.process(customer(env=env, name=f"customer {cust_number}"))
        cust_number+=1

random.seed(2)

env=simpy.Environment()

env.process(customer_generator(env=env))

env.run(until=10*60) #until 10 minutes
```

```
... customer 1: Arrives at time 174.72
customer 1: Details entered at time: 204.72
customer 1: Cash retrieved at time: 264.72
customer 2: Arrives at time 348.46
customer 2: Details entered at time: 378.46
customer 3: Arrives at time 415.25
customer 2: Cash retrieved at time: 438.46
customer 3: Details entered at time: 445.25
customer 4: Arrives at time 485.43
customer 3: Cash retrieved at time: 505.25
customer 4: Details entered at time: 515.43
customer 4: Cash retrieved at time: 575.43
```

Code Explanation

- `import random`

Used to generate random arrival times for customers, which makes the simulation more realistic.

Customer Process

- `def customer(env, name) :`
- Defines a **customer process**. Each customer behaves independently but shares the same simulation clock (env).

Code Explanation

```
print(f'{name}: Arrives at time {env.now: .2f}')
```

- As soon as the process starts, it prints the arrival time.
- **env.now** - current simulation time
- **: .2f** - formats the time to 2 decimal places

Code Explanation

```
yield env.timeout(30)
```

- The customer spends 30 seconds entering details.
- The process pauses for 30 seconds.
- During this time, other processes (other customers) can run.

Code Explanation

```
print(f' {name}: Details entered at time: {env.now:  
.2f}')
```

-After 30 seconds, the customer finishes entering details.

```
yield env.timeout(60)
```

-The customer waits another 60 seconds to retrieve cash.

```
print(f' {name}: Cash retrieved at time: {env.now:  
.2f}')
```

-After the wait, the transaction is complete.

Code Explanation

Customer Generator (Arrival Process)

`def customer_generator(env) :`

- This is a continuous generator of customers.
- Think of it as the system that keeps producing new arrivals.

Code Explanation

```
cust_number = 1
```

- Initializes a counter to uniquely name each customer.
- **while True:**
- This loop runs forever, continuously creating customers.

Code Explanation

```
random_inter_arrival_time = random.uniform(1,3)*60
```

Generates a random arrival gap:

- `random.uniform(1,3)` → random number between 1 and 3 minutes
- `* 60` → converts minutes to seconds
- So, customers arrive every 60 to 180 seconds.

Code Explanation

```
yield env.timeout(random_inter_arrival_time)
```

- The generator waits for that random time before creating the next customer.
- `env.process(customer(env=env, name=f"customer {cust_number}"))`
- Creates and schedules a new customer process.
- Each customer runs independently.
- Multiple customers can overlap in time.

Code Explanation

```
cust_number += 1
```

- Increments the customer ID.

Code Explanation

Simulation Setup

```
env = simpy.Environment()
```

- Creates the simulation environment (clock + scheduler).

```
env.process(customer_generator(env=env))
```

- Starts the customer arrival process.
- Without this, no customers would be created.

Code Explanation

Simulation Setup

```
env.run(until=10*60)
```

- Runs the simulation for 10 minutes (600 seconds).
- The generator keeps producing customers during this time.
- Each customer continues their own timeline even if others arrive.

Constant Arrival Times

Constant arrival times

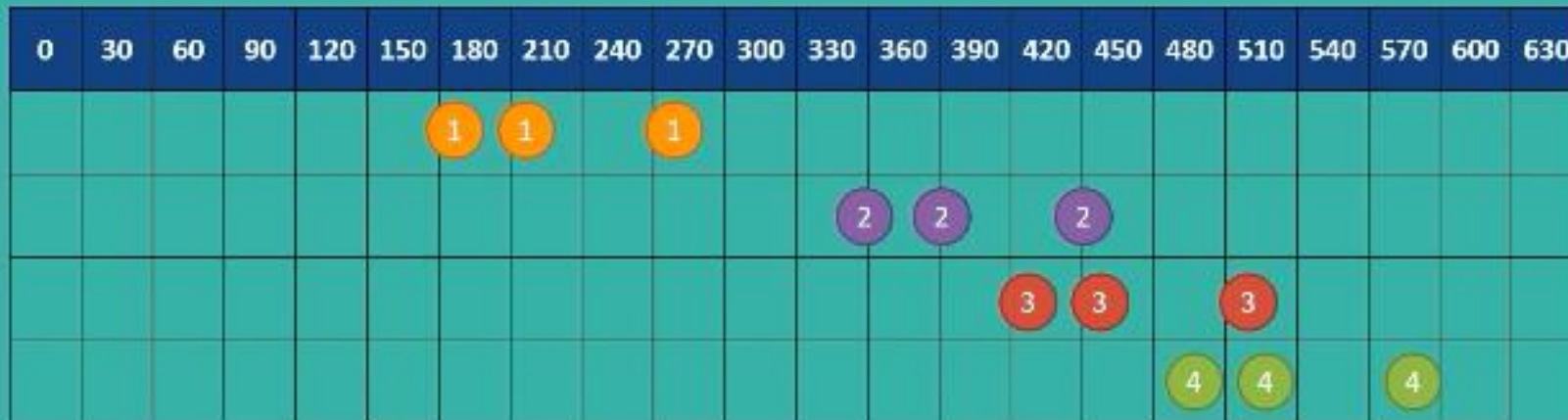
customer 1: Arrives at time 120
customer 1: Details entered by time 150
customer 1: Cash retrieved at time 210
customer 2: Arrives at time 240
customer 2: Details entered by time 270
customer 2: Cash retrieved at time 330
customer 3: Arrives at time 360
customer 3: Details entered by time 390
customer 3: Cash retrieved at time 450
customer 4: Arrives at time 480
customer 4: Details entered by time 510
customer 4: Cash retrieved at time 570



Random Arrival Time

Random arrival times

```
customer 1: Arrives at time 174.72  
customer 1: Details entered by time 204.72  
customer 1: Cash retrieved at time 264.72  
customer 2: Arrives at time 348.46  
customer 2: Details entered by time 378.46  
customer 3: Arrives at time 415.25  
customer 2: Cash retrieved at time 438.46  
customer 3: Details entered by time 445.25  
customer 4: Arrives at time 485.43  
customer 3: Cash retrieved at time 505.25  
customer 4: Details entered by time 515.43  
customer 4: Cash retrieved at time 575.43
```



Problem

- Customers don't wait for the existing customers to complete the service.



In reality, the customer has to wait



Code 1a

[21]

✓ 0s



```
import simpy
import random

def customer(env, name, atm):
    print(f'{name}: Arrives at time {env.now:.2f}')

    # Request access to the ATM
    with atm.request() as request:
        yield request # wait until ATM is available

    print(f'{name}: Starts using ATM at time {env.now:.2f}')

    yield env.timeout(30)
    print(f'{name}: Details entered at time: {env.now:.2f}')

    yield env.timeout(60)
    print(f'{name}: Cash retrieved at time: {env.now:.2f}')

    print(f'{name}: Leaves ATM at time {env.now:.2f}')

def customer_generator(env, atm):
    cust_number = 1

    while True:
        # Random arrival between 1-3 minutes
        inter_arrival = random.uniform(1, 3) * 60
        yield env.timeout(inter_arrival)

        env.process(customer(env, f"customer {cust_number}", atm))
        cust_number += 1

env = simpy.Environment()
```

Code 1b with Output

[21]

✓ 0s



```
env = simpy.Environment()
```

```
# Create ATM resource (capacity = 1 means only one customer at a time)
```

```
atm = simpy.Resource(env, capacity=1)
```

```
env.process(customer_generator(env, atm))
```

```
env.run(until=10 * 60)
```

▼

```
... customer 1: Arrives at time 168.79
customer 1: Starts using ATM at time 168.79
customer 1: Details entered at time: 198.79
customer 1: Cash retrieved at time: 258.79
customer 1: Leaves ATM at time 258.79
customer 2: Arrives at time 320.34
customer 2: Starts using ATM at time 320.34
customer 2: Details entered at time: 350.34
customer 2: Cash retrieved at time: 410.34
customer 2: Leaves ATM at time 410.34
customer 3: Arrives at time 475.11
customer 3: Starts using ATM at time 475.11
customer 3: Details entered at time: 505.11
customer 3: Cash retrieved at time: 565.11
customer 3: Leaves ATM at time 565.11
customer 4: Arrives at time 577.56
customer 4: Starts using ATM at time 577.56
```


Simulation Explanation

Random arrival times with resource

arrives # got ATM # entered details # retrieved cash

```
customer 1: Arrives at time 174.72
customer 1: gets ATM machine at time 174.72
customer 1: Details entered by time 204.72
customer 1: Cash retrieved at time 264.72
customer 2: Arrives at time 348.46
customer 2: gets ATM machine at time 348.46
customer 2: Details entered by time 378.46
customer 3: Arrives at time 415.25
customer 2: Cash retrieved at time 438.46
customer 3: gets ATM machine at time 438.46
customer 3: Details entered by time 468.46
customer 4: Arrives at time 485.43
customer 3: Cash retrieved at time 528.46
customer 4: gets ATM machine at time 528.46
customer 4: Details entered by time 558.46
```

